

Pembuatan PRD Detektor Transaksi

<https://gemini.google.com/app/e88753e638701d6d>

```
User prompt: import pandas as pd # ===== #
1. FORM INPUT UI COLAB (Klasifikasi) # (Ubah angka di panel sebelah kanan cell ini!) #
===== # @title 📊 Detektor Status
Untung/Rugi Sales_USD = 2000 # @param {type:"number"} Diskon_Persen = 40 # @param
{type:"slider", min:0, max:100, step:1} Biaya_Kirim_USD = 150 # @param {type:"number"}
Kuantitas = 5 # @param {type:"integer"} Kategori = 'Furniture' # @param ["Furniture", "Office
Supplies", "Technology"] Sub_Kategori = 'Tables' # @param ["Accessories", "Appliances", "Art",
"Binders", "Bookcases", "Chairs", "Copiers", "Envelopes", "Fasteners", "Furnishings", "Labels",
"Machines", "Paper", "Phones", "Storage", "Supplies", "Tables"] Segment = 'Corporate' #
@param ["Consumer", "Corporate", "Home Office"] Region = 'Southeast Asia' # @param
["Central", "North", "South", "East", "West", "Southeast Asia", "Oceania", "EMEA", "Africa",
"Canada", "Caribbean", "Central Asia", "North Asia"] Market = 'APAC' # @param ["APAC", "EU",
"US", "LATAM", "EMEA", "Africa", "Canada"] Ship_Mode = 'Standard Class' # @param
["Standard Class", "Second Class", "First Class", "Same Day"] Order_Priority = 'Medium' #
@param ["Low", "Medium", "High", "Critical"] # Masukkan ke dictionary (Ubah persentase
diskon jadi desimal) user_input_clf = { 'sales': [Sales_USD], 'discount': [Diskon_Persen /
100.0], 'quantity': [Kuantitas], 'shipping_cost': [Biaya_Kirim_USD], 'ship_mode':
[Ship_Mode], 'order_priority': [Order_Priority], 'segment': [Segment], 'market': [Market],
'region': [Region], 'category': [Kategori], 'sub_category': [Sub_Kategori] } #
===== # 2. PROSES PREDIKSI
(KLASIFIKASI) # ===== df_input_clf =
pd.DataFrame(user_input_clf) # Feature Engineering df_input_clf['shipping_cost_ratio'] =
df_input_clf['shipping_cost'] / df_input_clf['sales'] df_input_clf['discount_impact'] =
df_input_clf['discount'] * df_input_clf['sales'] df_input_clf['total_cost_spent'] = df_input_clf['sales']
+ df_input_clf['shipping_cost'] # Encoding & Penyesuaian Kolom df_input_encoded_clf =
pd.get_dummies(df_input_clf) training_columns_clf = X_train_scaled_clf.columns
df_input_encoded_clf = df_input_encoded_clf.reindex(columns=training_columns_clf,
fill_value=0) # Scaling menggunakan scaler_clf df_input_scaled_clf =
df_input_encoded_clf.copy() df_input_scaled_clf[num_cols_clf] =
scaler_clf.transform(df_input_encoded_clf[num_cols_clf]) # Prediksi Kelas (0 atau 1)
prediksi_status = xgb_classifier.predict(df_input_scaled_clf)[0] #
===== # 3. TAMPILAN OUTPUT #
=====
print("=====") print(f"🔗 OUTPUT MODEL:
{prediksi_status}") print("=====") if
prediksi_status == 1: print("✅ KESIMPULAN: UNTUNG (1) - Transaksi ini aman untuk
disetujui.") else: print("⚠️ KESIMPULAN: RUGI (0) - Peringatan! Transaksi ini berpotensi
merugikan perusahaan.") dari code ini bisa tolong buat prd nya
```

Response: Tentu, berdasarkan kode Google Colab yang kamu berikan, sistem yang dibangun adalah sebuah alat bantu keputusan berbasis Machine Learning untuk mendeteksi apakah sebuah transaksi penjualan berpotensi menguntungkan atau merugikan.

Berikut adalah draf **Product Requirements Document (PRD)** yang disusun berdasarkan alur logika kode tersebut.

Product Requirements Document (PRD): Detektor Status Untung/Rugi Transaksi

1. Ringkasan Eksekutif (Executive Summary)

Detektor Status Untung/Rugi adalah sebuah *tool* berbasis Machine Learning (menggunakan XGBoost) yang berfungsi untuk mengevaluasi parameter suatu transaksi penjualan (seperti nilai sales, diskon, biaya kirim, dan kategori produk) dan memprediksi apakah transaksi tersebut akan menghasilkan keuntungan (Profit) atau kerugian (Loss). Alat ini saat ini berjalan di lingkungan Google Colab menggunakan fitur *Form Input UI*.

2. Tujuan Produk (Objectives)

- **Mencegah Kerugian:** Memberikan sistem peringatan dini (*early warning system*) sebelum sebuah pesanan/transaksi disetujui.
- **Otomatisasi Keputusan:** Mempercepat proses evaluasi transaksi oleh tim *Sales* atau *Finance* menggunakan model prediktif, tanpa harus menghitung profitabilitas secara manual.

3. Pengguna Sasaran (Target Audience)

- **Tim Sales / Account Executive:** Menggunakan *tool* ini untuk mengecek apakah diskon besar yang mereka tawarkan ke klien akan menyebabkan kerugian perusahaan.
- **Tim Finance / Approval:** Digunakan sebagai lapis persetujuan akhir sebelum pesanan diproses.

4. Kebutuhan Fungsional (Functional Requirements)

4.1. Input Pengguna (User Inputs)

Sistem harus menyediakan antarmuka (Form UI Colab) yang memungkinkan pengguna memasukkan 11 variabel transaksi berikut:

Nama Variabel	Tipe Data	Deskripsi / Rentang Nilai
Sales_USD	Number	Total nilai penjualan dalam USD.
Diskon_Persen	Slider (0-100)	Persentase diskon yang diberikan (step: 1).
Biaya_Kirim_USD	Number	Biaya pengiriman dalam USD.
Kuantitas	Integer	Jumlah barang yang dibeli.
Kategori	Dropdown	Furniture, Office Supplies, Technology
Sub_Kategori	Dropdown	17 pilihan (mis. Tables, Chairs, Phones, dll.)
Segment	Dropdown	Consumer, Corporate, Home Office
Region	Dropdown	13 pilihan region (mis. Southeast Asia, Oceania, dll.)

Nama Variabel	Tipe Data	Deskripsi / Rentang Nilai
Market	Dropdown	APAC, EU, US, LATAM, EMEA, Africa, Canada
Ship_Mode	Dropdown	Standard Class, Second Class, First Class, Same Day
Order_Priority	Dropdown	Low, Medium, High, Critical

4.2. Pemrosesan Data (Data Processing & Feature Engineering)

Sebelum dilempar ke model prediktif, sistem harus memproses data mentah di *backend* (di dalam *script*):

1. **Konversi Diskon:** Mengubah nilai `Diskon_Persen` (0-100) menjadi desimal (0.0 - 1.0).
2. **Feature Engineering:** Sistem wajib membuat 3 kolom fitur baru:
 - `shipping_cost_ratio` = *Biaya Kirim* dibagi *Sales*
 - `discount_impact` = *Diskon (desimal)* dikali *Sales*
 - `total_cost_spent` = *Sales* ditambah *Biaya Kirim*
3. **Encoding:** Melakukan *One-Hot Encoding* (`pd.get_dummies`) pada variabel kategorikal dan menyesuaikan jumlah kolom dengan dataset *training* (`X_train_scaled_clf.columns`).
4. **Scaling:** Menstandarisasi variabel numerik menggunakan *scaler* yang sudah dilatih sebelumnya (`scaler_clf`).

4.3. Prediksi Model (Machine Learning Prediction)

- Sistem menggunakan model klasifikasi **XGBoost Classifier** (`xgb_classifier`).
- Model akan mengklasifikasikan data input ke dalam salah satu dari dua kelas: 1 atau 0.

4.4. Output Sistem (System Output)

Sistem harus menampilkan hasil prediksi secara jelas di layar output (konsol/terminal UI) dengan kriteria:

- **Jika Prediksi = 1:** Menampilkan pesan "  KESIMPULAN: UNTUNG (1) - Transaksi ini aman untuk disetujui."
- **Jika Prediksi = 0:** Menampilkan pesan "  KESIMPULAN: RUGI (0) - Peringatan! Transaksi ini berpotensi merugikan perusahaan."

5. Kebutuhan Non-Fungsional (Non-Functional Requirements)

- **Platform:** Saat ini harus dapat berjalan dengan lancar di environment Google Colaboratory.
- **Lantensi (Latency):** Waktu dari eksekusi cell hingga memunculkan output prediksi harus di bawah 2 detik.
- **Ketersediaan Model (Dependencies):** Sistem membutuhkan *library* Pandas, Scikit-Learn, dan XGBoost, serta *state* dari model, scaler, dan daftar kolom *training* yang sudah dimuat (*loaded*) sebelumnya ke memori *notebook*.

6. Pengembangan Lanjutan (Future Enhancements)

(Opsional - Ide untuk mengembangkan project)

1. **Migrasi Platform:** Memindahkan aplikasi dari Google Colab ke web framework seperti **Streamlit** atau **FastAPI** agar UI/UX lebih profesional dan dapat diakses tim non-teknikal tanpa melihat kode.
2. **Deteksi Regresi:** Selain memprediksi status Biner (Untung/Rugi), tambahkan model Regresi untuk memprediksi *estimasi nominal* profit/loss-nya secara presisi.
3. **Analisis What-If:** Fitur yang merekomendasikan: "Turunkan diskon menjadi 15% agar transaksi berubah menjadi status Untung."

```
User prompt: import pandas as pd # ===== #
1. FORM INPUT UI COLAB # (Silakan ubah angka di panel sebelah kanan cell ini!) #
===== # @title 🧮 Kalkulator Simulasi Profit
Sales_USD = 2000 # @param {type:"number"} Diskon_Persen = 25 # @param {type:"slider",
min:0, max:100, step:1} Biaya_Kirim_USD = 150 # @param {type:"number"} Kuantitas = 5 #
@param {type:"integer"} Kategori = 'Technology' # @param ["Furniture", "Office Supplies",
"Technology"] Sub_Kategori = 'Phones' # @param ["Accessories", "Appliances", "Art",
"Binders", "Bookcases", "Chairs", "Copiers", "Envelopes", "Fasteners", "Furnishings", "Labels",
"Machines", "Paper", "Phones", "Storage", "Supplies", "Tables"] Segment = 'Home Office' #
@param ["Consumer", "Corporate", "Home Office"] Region = 'Southeast Asia' # @param
["Central", "North", "South", "East", "West", "Southeast Asia", "Oceania", "EMEA", "Africa",
"Canada", "Caribbean", "Central Asia", "North Asia"] Market = 'APAC' # @param ["APAC", "EU",
"US", "LATAM", "EMEA", "Africa", "Canada"] Ship_Mode = 'Standard Class' # @param
["Standard Class", "Second Class", "First Class", "Same Day"] Order_Priority = 'Medium' #
@param ["Low", "Medium", "High", "Critical"] # Masukkan ke dictionary (Ubah persentase
diskon jadi desimal) user_input = { 'sales': [Sales_USD], 'discount': [Diskon_Persen /
100.0], 'quantity': [Kuantitas], 'shipping_cost': [Biaya_Kirim_USD], 'ship_mode':
[Ship_Mode], 'order_priority': [Order_Priority], 'segment': [Segment], 'market': [Market],
'region': [Region], 'category': [Kategori], 'sub_category': [Sub_Kategori] } #
===== # 2. PROSES PREDIKSI (Berjalan
otomatis saat Run) # ===== df_input =
pd.DataFrame(user_input) # Feature Engineering df_input['shipping_cost_ratio'] =
df_input['shipping_cost'] / df_input['sales'] df_input['discount_impact'] = df_input['discount'] *
df_input['sales'] df_input['total_cost_spent'] = df_input['sales'] + df_input['shipping_cost'] #
Encoding & Penyesuaian Kolom df_input_encoded = pd.get_dummies(df_input) #
training_columns = X_train_scaled.columns df_input_encoded =
df_input_encoded.reindex(columns=training_columns, fill_value=0) # Scaling df_input_scaled =
df_input_encoded.copy() df_input_scaled[num_cols] =
scaler.transform(df_input_encoded[num_cols]) # Prediksi prediksi_profit =
```

```
best_xgb.predict(df_input_scaled)
print("=====") print(f" 💰 ESTIMASI PROFIT:
USD {prediksi_profit[0]:.2f}") print("=====") if
prediksi_profit[0] < 0: print(" 🚨 PERINGATAN: Transaksi ini diprediksi MERUGIKAN!") else:
print(" ✅ STATUS AMAN: Transaksi menghasilkan keuntungan.") ini juga
```

Response: Tentu, kode kedua ini merupakan kelanjutan atau variasi dari sistem sebelumnya. Jika kode pertama adalah model **Klasifikasi** (hanya menebak status Untung/Rugi), kode kedua ini adalah model **Regresi** (memprediksi *nilai nominal* profit atau kerugiannya secara spesifik).

Berikut adalah draf **Product Requirements Document (PRD)** untuk Kalkulator Simulasi Profit.

Product Requirements Document (PRD): Kalkulator Simulasi Profit Transaksi

1. Ringkasan Eksekutif (Executive Summary)

Kalkulator Simulasi Profit adalah *tool* prediktif berbasis Machine Learning (menggunakan model regresi XGBoost) yang dirancang untuk mengestimasi besaran nilai Profit atau Loss (dalam USD) dari sebuah rencana transaksi. Alat ini memungkinkan pengguna untuk melakukan simulasi dengan mengubah variabel seperti diskon dan biaya kirim untuk melihat dampak langsungnya terhadap margin keuntungan sebelum pesanan difinalisasi.

2. Tujuan Produk (Objectives)

- **Estimasi Keuangan Presisi:** Memberikan proyeksi angka profitabilitas yang spesifik (bukan sekadar status aman/bahaya).
- **Simulasi Strategi Harga (Pricing Strategy):** Memungkinkan tim bisnis untuk mencoba berbagai skenario diskon (*what-if analysis*) guna menemukan titik optimal yang tetap menguntungkan.
- **Otomatisasi Kalkulasi:** Menggantikan perhitungan manual yang rumit dengan model prediktif yang sudah mempelajari pola data historis.

3. Pengguna Sasaran (Target Audience)

- **Pricing Strategist / Revenue Manager:** Untuk merancang batas maksimal diskon per kategori produk.
- **Tim Sales:** Sebagai kalkulator pintar sebelum mengajukan penawaran (*quotation*) ke klien.
- **Tim Finance:** Untuk memvalidasi margin keuntungan dari transaksi bervolume besar.

4. Kebutuhan Fungsional (Functional Requirements)

4.1. Input Pengguna (User Inputs)

Sistem menyediakan *Form UI* interaktif dengan 11 parameter transaksi. Parameter ini identik dengan sistem klasifikasi, namun difokuskan untuk simulasi angka:

Nama Variabel	Tipe Data	Deskripsi / Rentang Nilai
Sales_USD	Number	Total nilai penjualan dalam USD.
Diskon_Persen	Slider (0-100)	Persentase diskon yang diberikan (step: 1).
Biaya_Kirim_USD	Number	Biaya pengiriman dalam USD.
Kuantitas	Integer	Jumlah barang yang dibeli.
Kategori	Dropdown	Furniture, Office Supplies, Technology
Sub_Kategori	Dropdown	17 pilihan (mis. Phones, Tables, dll.)
Segment	Dropdown	Consumer, Corporate, Home Office
Region	Dropdown	13 pilihan region (mis. Southeast Asia, Oceania)
Market	Dropdown	APAC, EU, US, LATAM, EMEA, Africa, Canada
Ship_Mode	Dropdown	Standard Class, Second Class, First Class, Same Day
Order_Priority	Dropdown	Low, Medium, High, Critical

4.2. Pemrosesan Data (Data Processing & Feature Engineering)

Data input dari *form* akan ditransformasi melalui tahapan berikut:

1. **Konversi Diskon:** Mengubah parameter `Diskon_Persen` menjadi format desimal.
2. **Feature Engineering:** Membentuk 3 fitur turunan yang kritikal untuk prediksi regresi:
 - `shipping_cost_ratio` = `Biaya Kirim / Sales`
 - `discount_impact` = `Diskon (desimal) * Sales`
 - `total_cost_spent` = `Sales + Biaya Kirim`
3. **Encoding Kategorikal:** Mengubah variabel teks menjadi numerik dengan *One-Hot Encoding* (`pd.get_dummies`) dan menyelaraskan strukturnya dengan kolom data *training* (`X_train_scaled.columns`).
4. **Scaling Numerik:** Menormalisasi distribusi data menggunakan *scaler* bawaan (`scaler.transform`).

4.3. Prediksi Model (Machine Learning Prediction)

- Sistem menggunakan model **Regresi XGBoost** (`best_xgb`).

- Berbeda dengan sistem sebelumnya, model ini mengeluarkan output berupa **nilai kontinu** (angka desimal/float) yang merepresentasikan nominal estimasi profit dalam mata uang USD.

4.4. Output Sistem (System Output)

Sistem harus menampilkan hasil kalkulasi secara visual pada konsol dengan format:

1. **Angka Estimasi:** Menampilkan 💰 ESTIMASI PROFIT: USD [Nilai]. Nilai harus dibulatkan menjadi 2 angka di belakang koma (contoh: USD 150.50).
2. **Indikator Status:**
 - Jika output model < 0 (**Negatif**): Menampilkan pesan peringatan "🚨 PERINGATAN: Transaksi ini diprediksi MERUGIKAN!".
 - Jika output model ≥ 0 (**Positif**): Menampilkan pesan "✅ STATUS AMAN: Transaksi menghasilkan keuntungan."

5. Kebutuhan Non-Fungsional (Non-Functional Requirements)

- **Platform:** Berjalan di ekosistem Python (saat ini diimplementasikan via *Form UI* Google Colab).
- **Ketersediaan Lingkungan:** Bergantung pada *instance* di mana model `best_xgb`, parameter `num_cols`, dan `scaler` telah dilatih dan di-*load* ke dalam memori.
- **Performa:** Proses perhitungan dari manipulasi *dataframe* hingga inferensi model regresi harus diselesaikan dalam waktu kurang dari 2 detik.

6. Pengembangan Lanjutan (Future Enhancements)

1. **Integrasi Web dan Backend:** Sistem ini dapat dikemas ulang keluar dari Google Colab menjadi aplikasi web interaktif. Menggunakan Streamlit untuk *frontend* analitik, atau FastAPI jika ingin diekspos sebagai API yang dipanggil oleh aplikasi internal perusahaan.
2. **Deployment Fleksibel:** Model yang sudah di-*export* (misalnya menggunakan *joblib* atau *pickle*) dapat di-*deploy* ke *cloud hosting* seperti Vercel agar tim Sales bisa mengakses kalkulator ini melalui *smartphone* mereka tanpa perlu membuka *notebook* koding.
3. **Hybrid System:** Menggabungkan model Klasifikasi (Detektor) dan model Regresi (Kalkulator) dalam satu antarmuka UI. Jika model klasifikasi mendeteksi kerugian, model regresi otomatis memberitahu *berapa tepatnya* kerugian tersebut.